

Overview

Part 09 —  Model Context Protocol · 09-mcp/README.md

Module 09 — Model Context Protocol

Where you are: Stop 9 of 16 on the end-to-end AI track. **Time:** ~10–12 hours · **Prereq:** Module 04 — LLMs [04-llm] (tool calling) and Module 06 — AI Agents [06-ai-agents] (why a model needs tools at all).

Every AI application that connects a model to external tools and data used to write its own bespoke integration — an $M \times N$ problem: M applications, N systems, up to $M \times N$ one-off adapters, each maintained separately. MCP is Anthropic's answer, open-sourced in November 2024 and now governed as community infrastructure: one protocol, spoken by both server and client, so a tool or data source built once works with any MCP-speaking host. This module covers the protocol itself — its three primitives, its wire format, how to build a server well, and the trust boundary it creates the moment you connect one you didn't write. Module 13 — AI Security [13-ai-security] treats "MCP server as trust boundary" as a load-bearing prerequisite; this is where that idea gets built.

Learning objectives

By the end of this module you can:

1. Explain the $M \times N$ integration problem MCP solves and why the USB-C/LSP analogy is the right one to reach for.
2. Name the three MCP primitives (Tools, Resources, Prompts), state each one's control locus, and diagnose which primitive a given piece of functionality *should* be — and why using the wrong one is a real bug.
3. Draw the Host/Client/Server architecture from memory and explain why a host holds one client per server.
4. Trace a JSON-RPC 2.0 message exchange for both the classic (handshake-based) and current (stateless, `server/discover`-based) protocol eras, and say which one you'll actually meet in production today.
5. Build a minimal MCP server with well-named tools, load-bearing descriptions, and correct error handling (`ToolError` vs. a returned string) — and explain why the latter is the single most common bug.
6. Name the concrete MCP-specific attack classes (confused deputy, token passthrough, SSRF, tool poisoning, local server compromise) and the mitigation pattern for each.
7. Place MCP correctly against custom tool-calling, OpenAPI, and A2A — and defend when each is the right call.

Suggested path

#	Do this	File	Time
1	Primitives, architecture, wire protocol, building a server	notes/01-core-concepts.md	2h
2	Security model, ecosystem, MCP vs. alternatives	notes/02-security-ecosystem-and-alternatives.md	2h
3	Run a real MCP server (stdio)	code/minimal_mcp_server.py	0.5h
4	Run a real MCP client against it, read every printed message	code/mcp_client_demo.py	1h
5	Slides	slides/	0.5h
6	Lab exercises	lab/EXERCISES.md	4h
7	Papers & primary sources	papers/PAPERS.md	2h

Run the code with the repo venv:

```
/Users/srinip/ai-all/.venv/bin/python 09-mcp/code/mcp_client_demo.py
```

It launches `minimal_mcp_server.py` as a subprocess and prints every real JSON-RPC message exchanged — nothing here is mocked. No API key or network access required.

The 14 terms you must own

Host / Client / Server · Tool (model-controlled) · Resource (application-controlled) · Prompt (user-controlled) · JSON-RPC 2.0 · initialize handshake · stdio transport · Streamable HTTP transport · capability negotiation · tool description as interface · confused deputy · token passthrough · trust boundary · A2A (Agent2Agent)

Cross-links

- Module 04 — LLMs [04-llm] — tool-calling mechanics MCP standardizes the transport for.
- Module 06 — AI Agents [06-ai-agents] — tool design and the lethal trifecta; MCP is where "tool" as a concept gets a wire protocol.
- Module 07 — Agentic AI [07-agentic-ai] — MCP vs. A2A: agent-to-tool vs. agent-to-agent, and when each protocol is the right layer.
- Module 13 — AI Security [13-ai-security] — MCP servers as a trust boundary and a third-party dependency; prompt injection arriving through tool-returned data.
- Module 16 — Claude Code track [16-ai-tech-stack/tracks/claude-code] — the practical, day-to-day side of configuring and using MCP servers inside a real coding agent, which this module gives you the protocol underneath.

Exit check

You can hand a colleague `code/minimal_mcp_server.py` and `code/mcp_client_demo.py`, have them run the client, and then explain — pointing at the actual printed JSON-RPC messages, not from memory — which three methods correspond to which primitive, why the failed `search_catalog` call still returns `is_error=True`

rather than a protocol error, and what specific trust decision they'd need to make before pointing this same client at a third-party server they didn't write.